

DD: ILM-CFG-02 — Tech Region Registry

Developer implementation guide: DD_ILM-CFG-02-Tech_Region_Registry-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_ILM-CFG-02-Tech_Region_Registry-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Customer Service & Identity (companion to ILM-CFG-01) The system of record for **geographic service zones**: the operator's named regions, their boundaries (polygon / postcode / named-area), and the (region, task_type) → FOUNDATION_AUTH group code mapping used by Camunda + ICN-01 for dispatcher notification. Owned at operator scope (each operator defines its own zone topology). Referenced by FUL-02 STEP-CAPTURE for footprint validation and by WO module Drools rules for region resolution + candidate group lookup. ILM-CFG-02 is **purely structural-geographic**. Contractor identity, contractor service scope per region, contractor skills, contractor day/hour availability slots with concurrency caps, technician rosters, and skill certifications all live in **EM-02 (Employment Management, pending)** — never in this module. The boundary is strict: ILM-CFG-02 gives you the region; EM-02 tells you who can actually do work there.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_ILM-CFG-02-Tech_Region_Registry-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/tech-regions/{tech_region_id}/candidate-group-map/{taskType}
- GET /api/tech-regions/KE-NRB-LAVINGTON
- GET /api/tech-regions/{id}
- GET /api/tech-regions/{id}/candidate-group?taskType=X

- GET /api/tech-regions/{tech_region_id}
- GET /api/tech-regions/{tech_region_id}/candidate-group-map
- GET /api/tech-regions/{tech_region_id}/candidate-group?taskType=INSTALL
- GET /api/tech-regions?operatorCode=WIK&active=true
- GET /api/tech-regions?operatorCode=X
- PATCH /api/tech-regions/{tech_region_id}
- POST /api/em/contractor-availability
- POST /api/staff-notifications
- POST /api/tech-regions
- POST /api/tech-regions/check-footprint
- POST /api/tech-regions/resolve
- POST /api/work-orders/install
- PUT /api/tech-regions/{tech_region_id}/boundary
- PUT /api/tech-regions/{tech_region_id}/candidate-group-map/{taskType}

4.2 Tables

- tech_region
- tech_region_boundary
- tech_region_candidate_group_map

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- assign-contractor
- fiber-install
- ftth-cpe
- google-maps
- kenya-kyc-leaders
- kenya-l1-kyc
- vip-handling

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group I:** Foundations | **Version:** v1.0 | **Module:** Customer Service & Identity (companion to ILM-CFG-01)

The system of record for **geographic service zones**: the operator's named regions, their boundaries (polygon / postcode / named-area), and the (region, task_type) → FOUNDATION_AUTH group code mapping used by Camunda + ICN-01 for dispatcher notification. Owned at operator scope (each operator defines its own zone topology). Referenced by FUL-02 STEP-CAPTURE for footprint validation and by WO module Drools rules for region resolution + candidate group lookup.

ILM-CFG-02 is **purely structural-geographic**. Contractor identity, contractor service scope per region, contractor skills, contractor day/hour availability slots with concurrency caps, technician rosters, and skill certifications all live in **EM-02 (Employment Management, pending)** — never in this module. The boundary is strict: ILM-CFG-02 gives you the region; EM-02 tells you who can actually do work there.

1. What this module is

When a customer orders new fiber at "Karen Hardy, Karen, Nairobi", the system must first answer: *what geographic region is this address in, is it inside the operator's footprint, and which Camunda candidate group should be notified for the dispatcher task?* This module answers those geographic questions. It provides a structured registry of:

- **Tech regions** — the operator's named geographic zones (e.g., KE-NRB-KAREN, KE-NRB-EASTLANDS, KE-MOMBASA-NYALI).
- **Region boundaries** — polygons, postcodes, or named-area lists defining what's inside each region.
- **Candidate group codes** — Camunda BPMN-readable group codes (e.g., tech-region-NRB-KAREN) that map (region, task_type) to FOUNDATION_AUTH groups for ICN-01 dispatch.

The matching follow-up question — *which contractor is qualified and available to actually do the work in this region for the requested service scope, on the requested day, with the required skills?* — is answered by EM-02 (pending). ILM-CFG-02 provides the region identifier; EM-02 provides the scheduling answer.

What this module does

1. **Catalog tech regions** — operator-scoped named zones with stable IDs, display names, and parent regions (for hierarchy: country → province → city → neighborhood).
2. **Define region boundaries** — three representations supported: polygon (GeoJSON), postcode/zip code list, or named-area list. Operators pick the representation that matches their data quality.
3. **Address lookup** — given a service address (free text or structured), return the matching region(s). This is the entry point for WO routing and FUL-02 footprint validation.
4. **Footprint check** — is this address in our operator's footprint? — yes / no answer used by FUL-02 STEP-CAPTURE to gate order creation.
5. **Provide candidate group resolution** — given a tech region + a task type (install / shift / support / NOC / dispatcher), return the appropriate FOUNDATION_AUTH group code that ICN-01 + Camunda use for notification + task assignment.
6. **Track footprint lifecycle** — the operator may roll out new regions over time. Regions carry effective_from / effective_to for audit + retrospective routing analysis.

What this module does NOT do

- **Doesn't model contractors or technicians.** All entities around contractor identity, contractor service-scope per region, contractor technical skill sets, contractor availability slots (day/hour windows with concurrency caps), technician roster, skill certifications, and contractor performance live in EM-02 (pending). ILM-CFG-02 carries zero contractor or technician columns.
- **Doesn't route work orders.** Contractor selection — "which contractor covers KE-NRB-KAREN, has skill ftth-cpe, and has a free slot Monday at 14:00 with VIP-handling certification" — is answered by EM-02. The Drools assign-contractor rule in WO module queries EM-02, not ILM-CFG-02.
- **Doesn't track technician location or dispatch state.** That's WO module's domain (the WO row carries the assigned contractor + status). ILM-CFG-02 does not track who's in which region or where any person is.
- **Doesn't define the operator's coverage policy.** The polygon, postcode list, or named-area list is operator-supplied. We store it and look up against it; we don't decide what's a covered area.
- **Doesn't replace FOUNDATION_AUTH.** Candidate groups are FOUNDATION_AUTH groups; this module records the mapping (tech_region, task_type) → candidate_group_code but doesn't own group membership. A user being IN the FOUNDATION_AUTH group tech-region-NRB-KAREN is a FOUNDATION_AUTH concern; this module just says "for the NRB-KAREN region, install tasks notify this group code."
- **Doesn't do geocoding.** When given a free-text address, the lookup may use an external geocoding service (Google Maps Geocoding API or HERE, configured per operator) to resolve to lat/lng + structured components. The cached geocode result is stored on the customer's account, not in this module. ILM-CFG-02 just consumes the geocoded result.

Regional flexibility

Different operators have different zone-naming conventions and different coverage models:

- **WIK (Kenya)** — uses neighborhood-named regions (KE-NRB-KAREN, KE-NRB-EASTLANDS, KE-MOMBASA-NYALI, etc.). Hybrid coverage: some neighborhoods are direct internal teams, others are franchise contractors. Postal codes are sparse in Kenya; region boundaries primarily as named-area lists (with polygon fallback for high-detail regions).

- **WUG (Uganda)** — uses district-named regions (UG-KAMPALA-CENTRAL, UG-KAMPALA-NAKAWA, UG-ENTEBBE, etc.). Mostly franchise contractors.
- **WTZ (Tanzania)** — uses postcode + region hybrid (TZ-DSM-1101, TZ-DSM-1102, etc.). Postcode-based boundary representation is primary.
- **YASSN (Senegal)** — uses arrondissement + commune (SN-DKR-PLATEAU, SN-DKR-MEDINA, etc.). Polygon-based boundaries (better address data in Dakar). Multi-language: zones display in both French and Wolof labels where used.

The schema accommodates all four via:

- `region_boundary_type` enum (POLYGON_GEOJSON | POSTCODE_LIST | NAMED_AREA_LIST) — operator picks per region
- Multi-slot display labels (`display_name_primary`, `display_name_secondary`) for bilingual operators
- Per-operator `region_code_prefix` convention (KE, UG, TZ, SN) baked into stored IDs but not validated by ILM-CFG-02
- Per-operator candidate-group-code naming convention (e.g., WIK uses `tech-region-<code>`; YASSN uses `tech-zone-<code>`) configured per operator

Regional flexibility is intentional throughout: the boundary representation has 3 options and the candidate-group naming is per-operator config. New operators' conventions land via config, not DDL.

Glossary

Term	Meaning					
Tech region	A named operator-defined geographic zone, the unit of geographic identification. Example: KE-NRB-KAREN. May have parent regions (KE-NRB-KAREN parent is KE-NRB, which parents KE). Contractor coverage and technician scheduling against this region are EM-02 concerns; ILM-CFG-02 just provides the region identity.					
Region boundary	The geographic extent of a tech region, stored in one of three forms: GeoJSON polygon, postcode list, or named-area string list. ILM-CFG-02 uses this to answer "does service address X fall in this region?".					
Contractor	A company (or internal team) providing field services. Owned by EM-02. Referenced from EM-02 rows that anchor to <code>tech_region_id</code> . ILM-CFG-02 itself has no contractor table.					

Term	Meaning					
Technician	An individual person employed by a contractor. Owned by EM-02. ILM-CFG-02 has no technician table.					
Candidate group code	The FOUNDATION_AUTH group code used by Camunda BPMN to declare task assignees. ILM-CFG-02 maps (tech_region, task_type) → group_code. ICN-01 dispatches notifications to all members of that group.					
Footprint	The union of all currently-active (effective_to IS NULL OR > NOW()) tech regions for an operator. "Is this address in our footprint?" = "does it fall within at least one active region?".					
Task type	The kind of work to be done in a region: INSTALL	SHIFT	SUPPORT	RECOVERY	AUDIT	DISPATCHER. Different task types may map to different candidate groups within the same region (an installer and a support staff member may be different people).

2. Tables overview + relationships

2.1 Tables overview

ILM-CFG-02 owns three tables — purely structural-geographic. **Contractor identity, contractor service-scope per region, contractor skills, contractor availability slots, technician rosters, and skill certifications all live in EM-02 (Employment Management, pending).** The link from EM-02 entities back to ILM-CFG-02 is via tech_region_id references on EM-02-owned rows; ILM-CFG-02 itself carries no contractor or technician data.

Table	Purpose
tech_region	Operator-scoped catalog of named geographic zones, with hierarchy + display labels.
tech_region_boundary	Per-region boundary definition; one row per region, one of three representations (polygon, postcode list, named area).
tech_region_candidate_group_map	The (tech_region, task_type) → FOUNDATION_AUTH group_code mapping for Camunda + ICN-01 dispatch.

Full DDL in §3.

What EM-02 owns (not in this DD)

For context, the EM-02 module (pending) will own:

- **Contractor entity** — the company/franchise/internal-team record. Identity, contract terms, billing, performance metrics, address.
- **Contractor region+scope coverage** — which contractor handles which tech_region_id for which service_scope (INSTALL / SUPPORT / MAINTENANCE / RECOVERY / AUDIT). A contractor may handle INSTALL+SUPPORT in one region but only SUPPORT in another.

- LM-CFG-02 references EM-02 indirectly: WO module's Drools rules query EM-02 for "contractors available in KE-NRB-KAREN for service_scope=INSTALL on Monday 14:00 with skill ftth-cpe", and EM-02 in turn references tech_region_id to filter. ILM-CFG-02 does not query EM-02 itself.

```

+-----+
| tech_region | ← parent_region_id (self-ref hierarchy)
| (operator-scoped) |
+-----+
|
+-----+ +-----+
| v | | v |
+-----+ +-----+
| tech_region_ | | tech_region_ |
| boundary | | candidate_group_map |
| | | |
| 1 row per region | | (region, task_type) |
| + boundary type | | → FOUNDATION_AUTH |
| + data column | | group_code |
+-----+ +-----+

===== EM-02 boundary (everything below pending) =====

+-----+ +-----+ +-----+
| contractor | ■■■■ contractor_region_scope ■ | technician_staff ■
| (EM-02) | | + skills_per_region ■ | (EM-02) |
| | | ■ + availability_slots | | |
+-----+ +-----+ +-----+
| | ■ FK: tech_region_id ■
| | ■ → ILM-CFG-02 ■
+-----+

```

Only the top half is ILM-CFG-02. Everything below the EM-02 boundary line is owned by the future EM-02 DD. EM-02-owned rows reference tech_region_id to indicate which region(s) a contractor's scope+skills+slots apply to; ILM-CFG-02 doesn't reciprocate (no FK from tech_region to EM-02).

- **FUL-02 STEP-CAPTURE** calls `POST /api/tech-regions/check-footprint` to validate a customer's service address is in footprint before allowing order capture (the workshop's "is this a covered area?" gate).
- **WO module FLOW-INSTALLATION / FLOW-SHIFTING / FLOW-SUPPORT** queries `ILM-CFG-02` for region resolution + candidate group code, then queries **EM-02** for the actual contractor selection (which contractor has scope `INSTALL` in this region, has the required skills, has an available slot at the requested datetime). `ILM-CFG-02` provides the region identifier; `EM-02` provides the scheduling answer.
- **ICN-01** does NOT query this module directly — it queries `FOUNDATION_AUTH` for group members. But `ICN-01`'s candidate-group codes (e.g., `tech-region-NRB-KAREN`) `ORIGINATE` here in `tech_region_candidate_group_map`. Camunda BPMN authors look up the right group code from this module's tables when designing per-operator BPMNs.
- **ILM-CFG-01 (Customer Service)** `customer_account` rows can carry an optional `home_tech_region_id` denormalized field (resolved at order time, stored for fast access during support calls). This module is the source of truth; the account row is a cached pointer.

3. Schemas

3.1 tech_region

```
CREATE TABLE tech_region (
  tech_region_id      TEXT PRIMARY KEY,          -- 'KE-NRB-KAREN', 'SN-DKR-PLATEAU' — human-readable operator-s
  operator_code       TEXT NOT NULL,
  parent_region_id    TEXT NULL REFERENCES tech_region(tech_region_id), -- hierarchy: 'KE-NRB-KAREN' → 'KE-NRB' → 'KE
  display_name_primary TEXT NOT NULL,            -- 'Karen'
  display_name_secondary TEXT NULL,              -- bilingual; YASSN 'Plateau' / 'Pulako'
  region_type         TEXT NOT NULL,             -- 'COUNTRY' | 'PROVINCE' | 'CITY' | 'NEIGHBORHOOD' | 'CUSTOM'
  active              BOOLEAN NOT NULL DEFAULT TRUE,
  notes               TEXT NULL,
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  effective_from      DATE NOT NULL,             -- when this region became live
  effective_to        DATE NULL,                 -- NULL = ongoing
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by          TEXT NOT NULL
);

CREATE INDEX idx_tech_region_operator ON tech_region (operator_code, active);
CREATE INDEX idx_tech_region_parent  ON tech_region (parent_region_id);
CREATE INDEX idx_tech_region_type    ON tech_region (operator_code, region_type);
```

Sample data:

tech_region_id	operator_code	parent_region_id	display_name_primary	display_name_secondary	region_type	active	effective_from
KE	WIK	NULL	Kenya	NULL	COUNTRY	true	2010-01-01
KE-NRB	WIK	KE	Nairobi	NULL	CITY	true	2010-01-01
KE-NRB-KAREN	WIK	KE-NRB	Karen	NULL	NEIGHBORHOOD	true	2010-01-01
KE-NRB-LAVINGTON	WIK	KE-NRB	Lavington	NULL	NEIGHBORHOOD	true	2010-01-01
KE-NRB-EASTLANDS	WIK	KE-NRB	Eastlands	NULL	NEIGHBORHOOD	true	2010-01-01
KE-NRB-KIBERA	WIK	KE-NRB	Kibera	NULL	NEIGHBORHOOD	true	2015-06-01
KE-MOMBASA	WIK	KE	Mombasa	NULL	CITY	true	2012-03-01
KE-MOMBASA-NYALI	WIK	KE-MOMBASA	Nyali	NULL	NEIGHBORHOOD	true	2012-03-01
KE-DIANI	WIK	KE	Diani	NULL	CUSTOM	true	2018-10-01
UG	WUG	NULL	Uganda	NULL	COUNTRY	true	2011-05-01
UG-KAMPALA	WUG	UG	Kampala	NULL	CITY	true	2011-05-01
UG-KAMPALA-CENTRAL	WUG	UG-KAMPALA	Central Division	NULL	NEIGHBORHOOD	true	2011-05-01
UG-KAMPALA-NAKAWA	WUG	UG-KAMPALA	Nakawa Division	NULL	NEIGHBORHOOD	true	2011-05-01
TZ	WTZ	NULL	Tanzania	NULL	COUNTRY	true	2013-08-01
TZ-DSM	WTZ	TZ	Dar es Salaam	NULL	CITY	true	2013-08-01
TZ-DSM-1101	WTZ	TZ-DSM	Kinondoni 1101	NULL	CUSTOM	true	2013-08-01
SN	YASSN	NULL	Senegal	Sénégal	COUNTRY	true	2019-09-01
SN-DKR	YASSN	SN	Dakar	Ndakaaru	CITY	true	2019-09-01
SN-DKR-PLATEAU	YASSN	SN-DKR	Plateau	Pulako	NEIGHBORHOOD	true	2019-09-01
SN-DKR-MEDINA	YASSN	SN-DKR	Médina	Medina	NEIGHBORHOOD	true	2019-09-01

Row 9 (KE-DIANI) shows the CUSTOM region type — Diani is a tourist beach area that doesn't fit the standard city/neighborhood hierarchy but is a meaningful service zone for the operator's seasonal-customer business (per ILM-CFG-01 §2.3 multi-account customers).

Row 17-20 show YASSN's bilingual labels: `display_name_secondary` carries the Wolof translation where applicable.

3.2 tech_region_boundary

```
CREATE TABLE tech_region_boundary (
```

```

tech_region_id      TEXT PRIMARY KEY REFERENCES tech_region(tech_region_id),
boundary_type       TEXT NOT NULL,
boundary_polygon     JSONB NULL,
boundary_postcodes  TEXT[] NULL,
boundary_named_areas TEXT[] NULL,
source              TEXT NOT NULL,
confidence           TEXT NOT NULL,
last_validated_at   TIMESTAMP NULL,
notes               TEXT NULL,
updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
updated_by          TEXT NOT NULL,
CHECK ((boundary_type = 'POLYGON_GEOJSON' AND boundary_polygon IS NOT NULL) OR
       (boundary_type = 'POSTCODE_LIST' AND boundary_postcodes IS NOT NULL) OR
       (boundary_type = 'NAMED_AREA_LIST' AND boundary_named_areas IS NOT NULL))
);

CREATE INDEX idx_boundary_type ON tech_region_boundary (boundary_type);
-- GiST index on the polygon geometry would live here in production; v1.0 ships with B-tree on tech_region_id only

```

Sample data:

tech_region_id	boundary_type	boundary_polygon (excerpt)	boundary_postcodes	boundary_named_areas	source	confidence
KE-NRB-KAREN	NAMED_AREA_LIST	NULL	NULL	{Karen Hardy, Karen Plains, Karen Country Club, Karen Hills, Acacia Park, Karen C}	MANUAL	MEDIUM
KE-NRB-LAVINGTON	NAMED_AREA_LIST	NULL	NULL	{Othaya Road, Muthangari, Mageta Road, James Gichuru, Riverside Drive (west of Waiyaki Way)}	MANUAL	HIGH
KE-NRB-EASTLANDS	POLYGON_GEOJSON	{"type": "Polygon", "coordinates": [[[36.876, -1.290], [36.876, -1.310], [36.910, -1.310], [36.910, -1.290], [36.876, -1.290]]]}	NULL	NULL	GIS-IMPORT	HIGH
KE-MOMBASA-NYALIKI	POLYGON_GEOJSON	{"type": "Polygon", "coordinates": [[[39.6905, -4.011], [39.6905, -4.030], [39.7140, -4.030], [39.7140, -4.011], [39.6905, -4.011]]]}	NULL	NULL	GIS-IMPORT	HIGH
KE-DIANI	NAMED_AREA_LIST	NULL	NULL	{Diani Beach, Galu Beach, Tiwi Beach, Ukunda}	CONTRACTOR-DECLARED	LOW
TZ-DSM-1101	POSTCODE_LIST	NULL	{11101, 11102, 11103}	NULL	GIS-IMPORT	HIGH
SN-DKR-PLATEAU	POLYGON_GEOJSON	{"type": "Polygon", "coordinates": [[[-17.450, 14.6700], [-17.450, 14.6850], [-17.420, 14.6850], [-17.420, 14.6700], [-17.450, 14.6700]]]}	NULL	NULL	GIS-IMPORT	HIGH
UG-KAMPALA-CENTRAL	NAMED_AREA_LIST	NULL	NULL	{Nakasero, Old Kampala, Kampala Hill, Industrial Area, Kololo (Central side)}	MANUAL	MEDIUM

KE-NRB-KAREN uses named-area list because Karen's address data is sparse and polygon-based lookup would miss many real addresses. KE-NRB-EASTLANDS and KE-MOMBASA-NYALI use polygons because GIS data exists for them. KE-DIANI is LOW confidence + contractor-declared because the operator's data there is patchy — the boundary will be re-validated as part of footprint cleanup.

3.3 tech_region_candidate_group_map

This is the table ICN-01 and Camunda BPMN authors consult to find the right FOUNDATION_AUTH group code for a given (tech_region, task_type) combination.

```
CREATE TABLE tech_region_candidate_group_map (
  tech_region_id      TEXT NOT NULL REFERENCES tech_region(tech_region_id),
  task_type           TEXT NOT NULL,
  candidate_group_code TEXT NOT NULL,
  notes               TEXT NULL,
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by          TEXT NOT NULL,
  PRIMARY KEY (tech_region_id, task_type)
);

CREATE INDEX idx_cgm_group_code ON tech_region_candidate_group_map (candidate_group_code);
```

Sample data:

tech_region_id	task_type	candidate_group_code	notes
KE-NRB-KAREN	INSTALL	tech-region-NRB-KAREN-install	Internal team handles installs
KE-NRB-KAREN	SHIFT	tech-region-NRB-KAREN-install	Same team handles shifts
KE-NRB-KAREN	SUPPORT	tech-region-NRB-KAREN-support	Separate support roster
KE-NRB-KAREN	DISPATCHER	tech-region-NRB-KAREN	Top-level dispatcher group (matches the group code used in ICN-01 sample data §3.4)
KE-NRB-LAVINGTON	INSTALL	tech-region-NRB-LAVINGTON-install	Franchise A primary
KE-NRB-LAVINGTON	SUPPORT	tech-region-NRB-LAVINGTON-support	
KE-NRB-LAVINGTON	DISPATCHER	tech-region-NRB-LAVINGTON	
KE-NRB-EASTLANDS	INSTALL	tech-region-NRB-EASTLANDS-install	
KE-NRB-EASTLANDS	DISPATCHER	tech-region-NRB-EASTLANDS	
KE-NRB-KIBERA	INSTALL	tech-region-NRB-KIBERA-install	
KE-MOMBASA-NYALI	INSTALL	tech-region-MSA-NYALI-install	Mombasa-prefixed code
KE-DIANI	INSTALL	tech-region-MSA-NYALI-install	Same group as Nyali (shared contractor + roster)
SN-DKR-PLATEAU	INSTALL	tech-zone-DKR-PLATEAU-install	YASSN uses tech-zone- prefix per operator convention
SN-DKR-PLATEAU	DISPATCHER	tech-zone-DKR-PLATEAU	

Note KE-DIANI shares the same INSTALL group code as KE-MOMBASA-NYALI because the same contractor + roster cover both regions. This is the small-operations consolidation pattern — one Drools rule output (group code) maps to many regions if appropriate. YASSN uses a different prefix convention (tech-zone-) — captured per row, no enforcement.

The DISPATCHER task type yields a top-level group code (e.g., tech-region-NRB-KAREN) which matches the code already used in ICN-01 sample data (§3.4 row 3 of staff_notification showed candidate_group = 'tech-region-NRB-KAREN'). This is intentional: dispatcher notifications are the high-level "we need to assign a tech in this region" alerts; install/support/etc. are specific role notifications.

4. Rules

Rule	Description		
R-ILM-TR-1	A tech region's <code>tech_region_id</code> is operator-scoped and human-readable, conventionally <COUNTRY>-<CITY>-<NEIGHBORHOOD> with operator-defined prefix conventions. The ID is stable across rebrandings (display labels can change without renaming the ID).		
R-ILM-TR-2	A tech region with <code>active=false</code> does NOT participate in address-lookup matching or candidate-group resolution. Its data is retained for audit + retrospective queries. Re-activation requires explicit BO admin action.		
R-ILM-TR-3	<code>tech_region_boundary.boundary_type</code> determines which of the three boundary columns is populated; the CHECK constraint enforces exactly one is non-NULL. Switching boundary type for a region (e.g., upgrading from <code>NAMED_AREA_LIST</code> to <code>POLYGON_GEOJSON</code> after a GIS data import) is a full row update — the previous boundary representation is overwritten (no history table v1.0; future enhancement).		
R-ILM-TR-4	Address resolution policy: <code>POST /api/tech-regions/resolve</code> accepts (lat, lng) and/or structured address components. The resolver tries: (1) point-in-polygon for <code>POLYGON_GEOJSON</code> regions if lat/lng provided; (2) postcode exact-match for <code>POSTCODE_LIST</code> regions if structured address provides postcode; (3) named-area substring-match for <code>NAMED_AREA_LIST</code> regions if structured address provides locality/neighbourhood. Returns all matching regions ordered by <code>region_type</code> specificity (NEIGHBORHOOD before CITY before COUNTRY). Multiple matches at the same specificity are returned as a list — caller (FUL-02, WO) decides which to use.		
R-ILM-TR-5	The candidate group map (<code>tech_region_id, task_type</code>) → <code>candidate_group_code</code> is the single source of truth for Camunda BPMN authors. Per-operator naming conventions (tech-region- for KE; tech-zone- for YASSN) are NOT validated — operators bring their own conventions. The code only needs to match an existing <code>FOUNDATION_AUTH</code> group; verification happens at Camunda deploy time, not in this module.		
R-ILM-TR-6	Multiple regions can share the same <code>candidate_group_code</code> (e.g., KE-DIANI and KE-MOMBASA-NYALI share tech-region-MSA-NYALI-install because the same EM-02-managed contractor covers both). This is intentional and supports operator consolidation. The reverse — one region → multiple <code>candidate_group_codes</code> for the same <code>task_type</code> — is NOT supported; the PRIMARY KEY (region, task_type) enforces uniqueness.		

Rule	Description		
R-ILM-TR-7	The <code>home_tech_region_id</code> field on <code>customer_account</code> (per ILM-CFG-01 §2.3) is a denormalized pointer set at order time. If a region is deactivated or boundaries change, existing <code>customer_account</code> rows are NOT auto-updated — they remain pointing to whatever region was assigned. A background reconciliation job (out of v1.0 scope) can be added to handle bulk re-resolution.		
R-ILM-TR-8	Footprint validation: <code>POST /api/tech-regions/check-footprint</code> returns 200 with <code>inFootprint:true</code> if the address resolves to ANY active region; 200 with <code>inFootprint:false</code> otherwise. Called by FUL-02 STEP-CAPTURE to gate order creation. The check is operator-scoped — addresses outside the operator's regions return <code>inFootprint:false</code> even if they're in another operator's footprint.		
R-ILM-TR-9	Geocoding: when address resolution requires lat/lng but only structured address is supplied, the resolver MAY invoke an external geocoder. Geocoder choice is operator-config (<code>geocoder_provider</code> per operator: <code>google-maps</code>)	here	none). Result is NOT cached in ILM-CFG-02 — caching happens at the <code>customer_account</code> level (FUL-02 STEP-CAPTURE stores the resolved coords on the account record). Subsequent address-lookups for the same account skip geocoding.
R-ILM-TR-10	ILM-CFG-02 carries NO contractor or technician data. Queries about "who can do INSTALL in KE-NRB-KAREN on Monday at 14:00 with skill <code>fitth-cpe</code> " are routed to EM-02 (pending), which queries ILM-CFG-02 only for region resolution and uses its own contractor <code>scope+skill+availability+staff</code> model to answer. ILM-CFG-02 must never grow contractor/staff columns.		

5. REST API

5.1 Region catalog

```
GET /api/tech-regions?operatorCode=WIK&active=true HTTP/1.1
Authorization: Bearer <staff-or-service-jwt>
```

→ 200 OK

```
{
  "regions": [
    {
      "techRegionId":      "KE-NRB-KAREN",
      "operatorCode":      "WIK",
      "parentRegionId":    "KE-NRB",
      "displayNamePrimary": "Karen",
      "displayNameSecondary": null,
      "regionType":        "NEIGHBORHOOD",
      "active":             true,
      "effectiveFrom":      "2010-01-01",
      "effectiveTo":        null
    },
    {
      "techRegionId":      "KE-NRB-LAVINGTON",
      "operatorCode":      "WIK",
      "parentRegionId":    "KE-NRB",
      "displayNamePrimary": "Lavington",
      "regionType":        "NEIGHBORHOOD",
      "active":             true,
      "effectiveFrom":      "2010-01-01",
      "effectiveTo":        null
    },
    ...
  ],
  "totalCount": 19
}
```

```
}
```

Filters: parentRegionId, regionType, active, effectiveAt (date — return regions active on that date).

GET /api/tech-regions/{tech_region_id}	# detail incl. boundary, candidate group map
POST /api/tech-regions	# create new region (BO admin only)
PATCH /api/tech-regions/{tech_region_id}	# update mutable fields (display labels, active, notes)
PUT /api/tech-regions/{tech_region_id}/boundary	# set or update boundary

5.2 Address resolution

The primary read endpoint for FUL-02 + WO module.

```
POST /api/tech-regions/resolve HTTP/1.1
Authorization: Bearer <service-jwt>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "structuredAddress": {
    "city": "Nairobi",
    "neighborhood": "Karen Hardy",
    "postcode": null,
    "country": "KE"
  },
  "latLng": {
    "lat": -1.3289,
    "lng": 36.7062
  }
}

→ 200 OK
{
  "operatorCode": "WIK",
  "matches": [
    {
      "techRegionId": "KE-NRB-KAREN",
      "matchType": "NAMED_AREA",
      "matchDetail": "Karen Hardy matches named area in KE-NRB-KAREN",
      "specificity": "NEIGHBORHOOD",
      "active": true
    },
    {
      "techRegionId": "KE-NRB",
      "matchType": "HIERARCHICAL_PARENT",
      "matchDetail": "Parent of KE-NRB-KAREN",
      "specificity": "CITY",
      "active": true
    },
    {
      "techRegionId": "KE",
      "matchType": "HIERARCHICAL_PARENT",
      "matchDetail": "Parent of KE-NRB",
      "specificity": "COUNTRY",
      "active": true
    }
  ],
  "primaryMatchRegionId": "KE-NRB-KAREN"
}
```

primaryMatchRegionId is the most specific (lowest in hierarchy) match. If no neighborhood matches, falls back to the CITY-level match. If nothing matches, response is:

```
→ 200 OK
{
  "operatorCode": "WIK",
  "matches": [],
  "primaryMatchRegionId": null
}
```

5.3 Footprint check

Used by FUL-02 STEP-CAPTURE to gate order creation.

```
POST /api/tech-regions/check-footprint HTTP/1.1
Authorization: Bearer <ful-02-service-jwt>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "structuredAddress": {
    "city": "Naivasha",
```

```

    "neighborhood": "Lake View",
    "country":      "KE"
  },
  "latLng":        null
}
→ 200 OK
{
  "operatorCode":    "WIK",
  "inFootprint":     false,
  "checkedAt":       "2026-05-20T15:30:00Z",
  "reason":          "No active tech region matches 'Naivasha / Lake View'. KE-NRB and KE-MOMBASA and KE-DIANI regions c
}

```

Or when in footprint:

```

→ 200 OK
{
  "operatorCode":    "WIK",
  "inFootprint":     true,
  "primaryRegionId": "KE-NRB-KAREN",
  "checkedAt":       "2026-05-20T15:30:00Z"
}

```

5.4 Candidate group resolution

GET /api/tech-regions/{tech_region_id}/candidate-group?taskType=INSTALL HTTP/1.1
 Authorization: Bearer <service-jwt>

```

→ 200 OK
{
  "techRegionId":    "KE-NRB-KAREN",
  "taskType":        "INSTALL",
  "candidateGroupCode": "tech-region-NRB-KAREN-install",
  "expectedMemberCount": 3,
  "lastUpdated":     "2026-04-12T10:00:00Z"
}

```

expectedMemberCount is the count of members in the corresponding FOUNDATION_AUTH group (queried live from FOUNDATION_AUTH, with a 1-hour cache). Used by ICN-01 to size the expectedRecipients field at notification creation.

5.5 Candidate group map management (BO admin)

GET /api/tech-regions/{tech_region_id}/candidate-group-map # list all task_type → group_code mappings
 PUT /api/tech-regions/{tech_region_id}/candidate-group-map/{taskType} # set / update one mapping
 DELETE /api/tech-regions/{tech_region_id}/candidate-group-map/{taskType} # remove one mapping

Contractor coverage management, technician roster management, contractor scope/skill/availability — all owned by EM-02 (pending), exposed via that module's own API surface. Not in ILM-CFG-02's API.

6. Kafka events

ILM-CFG-02 publishes on `sophix.ilm.tr.*`. The events are informational — downstream consumers cache region data and need invalidation signals.

Event type	Topic	Emitted when	Consumers
TechRegionCreated	<code>sophix.ilm.tr.region-created</code>	New tech_region row	WO module, FUL-02, BO UI
TechRegionUpdated	<code>sophix.ilm.tr.region-updated</code>	Mutable fields changed on tech_region	WO module, FUL-02 cache invalidation
TechRegionDeactivated	<code>sophix.ilm.tr.region-deactivated</code>	<code>active=true → false</code>	WO module (re-route in-flight WOs), FUL-02 (block new orders)
TechRegionBoundaryUpdated	<code>sophix.ilm.tr.boundary-updated</code>	tech_region_boundary row changed	FUL-02 address-resolve cache invalidation
TechRegionCandidateGroupMapUpdated	<code>sophix.ilm.tr.cgm-updated</code>	Candidate group code changed	Camunda BPMN authors notified (BO admin ops dashboard); ICN-01 cache invalidation

6.1 Sample event — TechRegionDeactivated

```

{
  "eventType":    "TechRegionDeactivated",
  "aggregateId":  "KE-NRB-KIBERA",
  "timestamp":    "2026-05-20T14:00:00.000Z",
}

```

```

    "payload": {
      "operatorCode":      "WIK",
      "techRegionId":      "KE-NRB-KIBERA",
      "deactivatedAt":      "2026-05-20T14:00:00Z",
      "reason":             "Contractor relationship terminated; no replacement available",
      "openWorkOrdersInRegion": 7,
      "activeCustomerAccounts": 423,
      "actionRequired":      "Customer accounts with home_tech_region_id = KE-NRB-KIBERA require re-routing decision. Open
    }
  }
}

```

This event is high-impact — a region going dark affects customer service. The payload includes counts of impacted records so the consuming systems can prioritize their response.

7. Runtime walkthrough — narrative

Two scenarios: (a) FUL-02 STEP-CAPTURE checks footprint before order creation; (b) WO module FLOW-INSTALLATION resolves tech assignment.

7.1 Scenario A — FUL-02 footprint check at order capture

A door-to-door agent in Lavington opens the FUL-02 BO UI to capture a new order at "Othaya Road 14, Lavington, Nairobi", customer Hon. Sarah Wanjiku. The agent enters the address. The BO UI calls FUL-02 STEP-CAPTURE, which before persisting the order calls:

```

POST /api/tech-regions/check-footprint HTTP/1.1
Authorization: Bearer <ful-02-service-jwt>
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "structuredAddress": {
    "city":             "Nairobi",
    "neighborhood":     "Lavington",
    "country":          "KE"
  },
  "latLng":            { "lat": -1.2745, "lng": 36.7691 }
}

```

ILM-CFG-02's resolver:

1. Pulls all active tech_region rows for WIK.
2. For each region with boundary_type= 'POLYGON_GEOJSON': runs point-in-polygon for the lat/lng. (Spatial index would accelerate this in production.)
3. For each region with boundary_type= 'NAMED_AREA_LIST': substring-matches the address neighborhood against boundary_named_areas. "Lavington" matches KE-NRB-LAVINGTON (the region's display_name_primary also matches; both signals agree).
4. Returns KE-NRB-LAVINGTON as the primary match.

Response:

```

{
  "operatorCode":      "WIK",
  "inFootprint":       true,
  "primaryRegionId":   "KE-NRB-LAVINGTON",
  "checkedAt":         "2026-02-16T17:25:00Z"
}

```

FUL-02 stores KE-NRB-LAVINGTON on the order record's home_tech_region_id field, proceeds to KYC + payment steps. Per R-ILM-TR-12, the geocode result (lat/lng) is also stored on the order so future operations don't re-geocode.

7.2 Scenario B — WO module FLOW-INSTALLATION resolves contractor + tech assignment

Later, Hon. Wanjiku's KYC is approved and payment captured. FUL-02 calls WO module's POST /api/work-orders/install endpoint with accountId, home_tech_region_id = KE-NRB-LAVINGTON, install date preference (Monday 14:00–17:00), service_class_1 = VIP. WO module FLOW-INSTALLATION starts; its Drools assign-contractor rule fires and needs to pick a contractor.

The Drools rule first calls ILM-CFG-02 only to confirm the region is still active + to fetch the candidate group code:

```
GET /api/tech-regions/KE-NRB-LAVINGTON HTTP/1.1
Authorization: Bearer <wo-service-jwt>

→ 200 OK
{
  "techRegionId":      "KE-NRB-LAVINGTON",
  "operatorCode":      "WIK",
  "displayNamePrimary": "Lavington",
  "regionType":        "NEIGHBORHOOD",
  "active":            true,
  "effectiveFrom":     "2010-01-01"
}
```

Then the Drools rule calls EM-02 (pending) to find an available contractor with the right scope + skills + slot capacity:

```
POST /api/em/contractor-availability HTTP/1.1
Authorization: Bearer <wo-service-jwt>
Content-Type: application/json

{
  "operatorCode":      "WIK",
  "techRegionId":      "KE-NRB-LAVINGTON",
  "serviceScope":      "INSTALL",
  "requiredSkills":     ["fiber-install", "ftth-cpe", "vip-handling"],
  "windowStart":        "2026-02-23T14:00:00+03:00",
  "windowEnd":          "2026-02-23T17:00:00+03:00",
  "concurrentDemand":   1
}

→ 200 OK
{
  "availableContractors": [
    {
      "contractorId":    "wik-internal-tech-team",
      "contractorRole":  "INTERNAL",
      "matchedSkills":   ["fiber-install", "ftth-cpe", "vip-handling"],
      "slot":            { "day": "MONDAY", "start": "09:00", "end": "18:00", "capacityRemaining": 3 },
      "rank":            1
    },
    {
      "contractorId":    "contractor_ke_nairobi_a",
      "contractorRole":  "PRIMARY",
      "matchedSkills":   ["fiber-install", "ftth-cpe"],
      "slot":            { "day": "MONDAY", "start": "08:00", "end": "19:00", "capacityRemaining": 1 },
      "rank":            2
    }
  ]
}
```

EM-02's response carries everything WO needs: the contractor selected, the slot it's assigned against (which EM-02 decrements internally), the matched skills. The Drools rule picks rank 1 (internal team, full skill match including vip-handling for VIP customer Hon. Wanjiku).

WO module then writes the WO row with assigned_contractor_id = wik-internal-tech-team, parks the BPMN at the "dispatcher-confirm" user task, and Camunda's HTTP connector calls ICN-01:

```
POST /api/staff-notifications HTTP/1.1
{
  "operatorCode":      "WIK",
  "sourceModule":       "WO-01",
  "sourceTaskId":       "cam_task_disp_01HZ_WANJIKU",
  "candidateGroup":     "tech-region-NRB-LAVINGTON",
  ...
}
```

The candidate group tech-region-NRB-LAVINGTON was looked up from ILM-CFG-02's tech_region_candidate_group_map.candidate_group_code WHERE tech_region_id = 'KE-NRB-LAVINGTON' AND task_type = 'DISPATCHER'. ICN-01 then resolves group membership via FOUNDATION_AUTH and dispatches the notification per its rules (DD_ICN-01 §7.0).

The actual selection of which technician within the internal team handles the WO happens inside EM-02 + the internal team's own dispatcher workflow — ILM-CFG-02 sees none of it. The boundary is clean: ILM-CFG-02 provides region resolution + the candidate group code for dispatcher notification; EM-02 provides everything about who can actually do the work.

8. Cross-DD coordination

8.1 Upstream callers

Caller	Endpoint	When
FUL-02 STEP-CAPTURE	POST /api/tech-regions/check-footprint	Every new order, before any persistence
FUL-02 STEP-CAPTURE	POST /api/tech-regions/resolve	After footprint check, to store the primary region on the order
WO module FLOW-* (all)	GET /api/tech-regions/{id} + GET /api/tech-regions/{id}/candidate-group?taskType=X	At Drools assign-contractor rule firing — to confirm region is active + get the candidate group code for ICN-01 dispatch. The actual contractor selection then queries EM-02.
EM-02 (pending)	GET /api/tech-regions?operatorCode=X	EM-02 BO admin UI loads the region list when an operator user adds contractor coverage rows. EM-02 reads ILM-CFG-02; ILM-CFG-02 does not read EM-02.
CSR BO UI (ILM-CFG-01 consumer)	GET /api/tech-regions/{id}	When CSR opens an account, displays the home tech region name for context

8.2 Downstream consumers (Kafka)

Consumer	Events subscribed	Action
WO module	TechRegionDeactivated	Invalidate routing rule region cache; alert ops on in-flight WO impact
FUL-02	TechRegionDeactivated, TechRegionBoundaryUpdated	Invalidate address-resolve cache; block new orders in deactivated regions
ICN-01	TechRegionCandidateGroupMapUpdated	Invalidate expectedMemberCount cache
EM-02 (pending)	TechRegionDeactivated, TechRegionUpdated	Invalidate EM-02-side region cache; flag contractor coverage rows pointing at deactivated regions for review
BO admin ops dashboard	All sophix.ilm.tr.*	Display region topology + change log
CVM	TechRegionDeactivated	Customer retention analytics for affected accounts

8.3 Sibling modules

Module	Direction	Contract
ILM-CFG-01 (Customer Service)	bidirectional	ILM-CFG-01's <code>customer_account.home_tech_region_id</code> denormalizes the primary tech region for fast UI display. ILM-CFG-02 emits <code>TechRegionDeactivated</code> so ILM-CFG-01 can flag affected accounts (out of scope for v1.0 — manual reconciliation).
EM-02 (pending)	ILM-CFG-02 → EM-02 (one-way reference)	EM-02 owns every entity around contractor identity, contractor scope per region, contractor skill set per region, contractor day/hour availability slots with concurrency caps, contractor employed technicians, skill catalog, and certifications. EM-02-owned rows reference <code>tech_region_id</code> to anchor coverage geographically. ILM-CFG-02 carries no contractor or technician columns. The two modules connect via the <code>tech_region_id</code> foreign key from EM-02 → ILM-CFG-02 only.
FOUNDATION_AUTH	ILM-CFG-02 ← AUTH (informational)	The <code>candidate_group_code</code> values in <code>tech_region_candidate_group_map</code> MUST correspond to existing FOUNDATION_AUTH groups; verification happens at Camunda deploy time, NOT in this module. ILM-CFG-02 trusts the BO admin to enter valid codes.
WO module	ILM-CFG-02 ← WO (read region) + EM-02 ← WO (read contractor availability)	WO module queries ILM-CFG-02 for region confirmation + candidate group code, then queries EM-02 for the actual contractor selection (scope + skills + slot availability). Caching at the WO module side (TTL ~5 min for region data, shorter for EM-02 slot data) is recommended.
FUL-02	ILM-CFG-02 ← FUL (read)	FUL-02 reads footprint + region resolution at order-creation time. Result is cached on the order row. FUL-02 does NOT query EM-02 at order capture — contractor selection happens later, at WO creation.

8.4 Dependencies (build order)

1. **ILM-CFG-01** (Customer Service) — already shipped; provides the `customer_account.home_tech_region_id` field.
2. **FOUNDATION_AUTH** — groups must exist before being referenced in `tech_region_candidate_group_map`. Operator deployment includes a `auth-groups-seed.sql` script creating the standard groups (`tech-region-*-install`, `*-support`, `*-DISPATCHER`, `kenya-l1-kyc`, `kenya-kyc-leaders`, etc.).
3. **Geocoding service** — operator deployment configures `geocoder_provider` (one of `google-maps`, `here`, `none`). If `none`, footprint check operates `polygon/postcode-only` without geocoder fallback for unstructured addresses.
4. **Region + boundary + candidate-group-map seed** — per-operator seed scripts populate the three tables from the operator's existing CRM/GIS data. WIK v1.0 seed includes all Nairobi neighborhoods + Mombasa + Diani; YASSN v1.0 seed includes Dakar arrondissements; WUG and WTZ have partial seeds (pending full operator data).
5. **EM-02 (pending — does NOT block ILM-CFG-02 v1.0)** — ILM-CFG-02 v1.0 ships and operates without EM-02. WO module routing rules can fall back to a flat config-file contractor list until EM-02 lands. ILM-CFG-02 itself has no FK dependency on EM-02.

9. Workshop sources

No dedicated workshop deep-dive for Tech Region Registry. The functional area is implicit in:

- **Work Order workshop** (deep-dive 07) — contractor dispatcher routing, technician assignment workflows.
- **HPRFS workshop** (deep-dive 03) — homepass-to-region relationships (a HomePass per RLM-CFG-01 is anchored to a physical address, which falls in a region).
- **Customer Management workshop** (deep-dive 01) — multi-location customers in Kenya, references to "Karen" / "Eastlands" / "Lavington" / "Diani" as service-zone-relevant areas.

V3 design intent for ILM-CFG-02: **provide a structured, queryable, operator-extensible geographic registry** that decouples routing logic from hard-coded region knowledge. The TZ codebase's current handling of tech regions is unverified pending cartography; placeholder: "TBD pending codebase review."

Out of v1.0 scope (handled by EM-02 pending):

- Contractor identity and registry.
- Contractor service scope per region (`INSTALL` / `SUPPORT` / `MAINTENANCE` / `RECOVERY` / `AUDIT`).
- Contractor skill set per region (`fiber-install`, `ftth-cpe`, `vip-handling`, ...).
- Contractor availability slots — day-of-week + hour-range windows with concurrency caps (e.g., "MONDAY 09:00–18:00, max 5 concurrent assignments").
- Contractor employed technician roster and skill certifications.

Out of v1.0 scope (other):

- **Real-time technician location tracking** — out (WO module's domain if/when implemented).
- **Spatial indexing optimization** — v1.0 ships with B-tree indexes only; GiST/PostGIS-backed point-in-polygon is a follow-up performance enhancement.
- **Automated boundary refresh from external GIS sources** — out; v1.0 boundaries are operator-loaded via seed scripts or BO admin manual entry.
- **Bulk customer_account.home_tech_region_id re-resolution job** — out; if a region's boundary changes, existing accounts retain their cached region reference. A future reconciliation job can be added.

End of DD_ILM-CFG-02-Tech_Region_Registry-v1.0.